

**Московский авиационный институт
(национальный исследовательский университет)**

**Факультет информационных технологий и прикладной
математики**

Кафедра вычислительной математики и программирования

Лабораторная работа №2 по курсу «Дискретный анализ»

Студент: И. А. Казаков
Преподаватель: А. А. Кухтичев
Группа: М8О-206БВ-24
Дата:
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №2

Задача: Необходимо создать программную библиотеку, реализующую структуру данных **PATRICIA tree**. На её основе разработать программу-словарь. Ключи — регистронезависимые строки (до 256 символов), значения — числа от 0 до $2^{64} - 1$.

Программа должна поддерживать:

- Добавление, поиск и удаление элементов.
- Сохранение и загрузку дерева в компактном бинарном виде.

Требуется провести исследование качества программы с использованием профилировщика **gprof** и средств анализа памяти.

1 Описание структуры данных

Дерево **PATRICIA** (Practical Algorithm to Retrieve Information Coded in Alphanumeric) представляет собой сжатое префиксное дерево. В отличие от обычного дерева, здесь узлы с одним потомком удаляются, а ветвление происходит по номеру критического бита (`bit_idx`), в котором ключи различаются. Это позволяет хранить ключи произвольной длины, не увеличивая высоту дерева пропорционально длине строк.

2 Исходный код

Реализация выполнена на языке C++.

```
1  #include <algorithm>
2  #include <cstdint>
3  #include <cstring>
4  #include <fstream>
5  #include <iostream>
6  #include <string>
7  #include <type_traits>
8
9  template <typename T>
10 concept Numeric = std::is_scalar_v<T>;
11
12 template <Numeric T>
13 class TrieNode {
14     public:
15         std::string k;
16         T val;
17         size_t bit_idx;
18         TrieNode *l_ptr, *r_ptr;
19
20         TrieNode(const std::string& key, T value, size_t idx)
21             : k(key), val(value), bit_idx(idx), l_ptr(nullptr), r_ptr(nullptr) {}
22
23         ~TrieNode() = default;
24 };
25
26 template <Numeric T>
27 class TrieMap {
28     private:
29         TrieNode<T>* head = nullptr;
30
31         uint8_t fetch_bit(const std::string& key, size_t bit_pos) {
32             if (bit_pos == 0) {
33                 return 0;
34             }
35             size_t char_idx = (bit_pos - 1) / 8;
```

```

36     if (char_idx >= key.length()) {
37         return 0;
38     }
39     return (static_cast<uint8_t>(key[char_idx]) >> (7 - (bit_pos - 1) % 8)) & 1;
40 }
41
42 size_t diff_at(const std::string& s1, const std::string& s2) {
43     size_t max_len = std::max(s1.length(), s2.length());
44     for (size_t i = 0; i < max_len; ++i) {
45         uint8_t b1 = (i < s1.length()) ? (uint8_t)s1[i] : 0;
46         uint8_t b2 = (i < s2.length()) ? (uint8_t)s2[i] : 0;
47         if (b1 != b2) {
48             uint8_t xor_res = b1 ^ b2;
49             int p = 7;
50             while (!(xor_res & (1 << p))) {
51                 p--;
52             }
53             return i * 8 + (7 - p) + 1;
54         }
55     }
56     return 0;
57 }
58
59 void deep_clear(TrieNode<T>* node) {
60     if (node->l_ptr && node->l_ptr->bit_idx > node->bit_idx) {
61         deep_clear(node->l_ptr);
62     }
63     if (node->r_ptr && node->r_ptr->bit_idx > node->bit_idx) {
64         deep_clear(node->r_ptr);
65     }
66     delete node;
67 }
68
69 void write_recursive(std::ofstream& os, TrieNode<T>* node) {
70     if (!node) {
71         return;
72     }
73     size_t sz = node->k.size();
74     os.write(reinterpret_cast<const char*>(&sz), sizeof(sz));
75     os.write(node->k.c_str(), sz);
76     os.write(reinterpret_cast<const char*>(&node->val), sizeof(T));
77
78     if (node->l_ptr && node->l_ptr->bit_idx > node->bit_idx) {
79         write_recursive(os, node->l_ptr);
80     }
81     if (node->r_ptr && node->r_ptr->bit_idx > node->bit_idx) {
82         write_recursive(os, node->r_ptr);
83     }
84 }

```

```

85
86 void locate(const std::string& key, TrieNode<T>*& target, TrieNode<T>*& parent,
87             TrieNode<T>*& grandparent) {
88     target = head->l_ptr;
89     parent = head;
90     grandparent = nullptr;
91     while (target->bit_idx > parent->bit_idx) {
92         grandparent = parent;
93         parent = target;
94         if (fetch_bit(key, target->bit_idx) == 0) {
95             target = target->l_ptr;
96         } else {
97             target = target->r_ptr;
98         }
99     }
100
101 public:
102     TrieMap() = default;
103
104     ~TrieMap() {
105         if (!head) {
106             return;
107         }
108         if (head->l_ptr != head) {
109             deep_clear(head->l_ptr);
110         }
111         delete head;
112     }
113
114     bool add(const std::string& key, T value) {
115         if (!head) {
116             head = new TrieNode<T>(key, value, 0);
117             head->l_ptr = head;
118             return true;
119         }
120
121         TrieNode<T>* prev = head;
122         TrieNode<T>* curr = head->l_ptr;
123         while (curr->bit_idx > prev->bit_idx) {
124             prev = curr;
125             if (fetch_bit(key, curr->bit_idx) == 0) {
126                 curr = curr->l_ptr;
127             } else {
128                 curr = curr->r_ptr;
129             }
130         }
131
132         if (key == curr->k) {

```

```

133         return false;
134     }
135
136     size_t new_idx = diff_at(key, curr->k);
137     prev = head;
138     curr = head->l_ptr;
139     while (curr->bit_idx > prev->bit_idx && curr->bit_idx < new_idx) {
140         prev = curr;
141         if (fetch_bit(key, curr->bit_idx) == 0) {
142             curr = curr->l_ptr;
143         } else {
144             curr = curr->r_ptr;
145         }
146     }
147
148     TrieNode<T>* leaf = new TrieNode<T>(key, value, new_idx);
149     uint8_t bit = fetch_bit(key, new_idx);
150     if (bit == 0) {
151         leaf->l_ptr = leaf;
152         leaf->r_ptr = curr;
153     } else {
154         leaf->l_ptr = curr;
155         leaf->r_ptr = leaf;
156     }
157
158     if (prev == head) {
159         prev->l_ptr = leaf;
160     } else {
161         if (fetch_bit(key, prev->bit_idx) == 0) {
162             prev->l_ptr = leaf;
163         } else {
164             prev->r_ptr = leaf;
165         }
166     }
167     return true;
168 }
169
170 T* find(const std::string& key) {
171     if (!head) {
172         return nullptr;
173     }
174     TrieNode<T>* p = head;
175     TrieNode<T>* c = head->l_ptr;
176     while (c->bit_idx > p->bit_idx) {
177         p = c;
178         if (fetch_bit(key, c->bit_idx) == 0) {
179             c = c->l_ptr;
180         } else {
181             c = c->r_ptr;

```

```

182     }
183 }
184 if (c->k == key) {
185     return &(c->val);
186 } else {
187     return nullptr;
188 }
189 }
190
191 bool del_key(const std::string& key) {
192     if (!head) {
193         return false;
194     }
195     if (key == head->k && head->l_ptr == head) {
196         delete head;
197         head = nullptr;
198         return true;
199     }
200
201     TrieNode<T>*target, *q, *parent_q;
202     locate(key, target, q, parent_q);
203
204     if (!target || target->k != key) {
205         return false;
206     }
207
208     if (target == q && target != head) {
209         TrieNode<T>* child;
210         if (target->l_ptr == target) {
211             child = target->r_ptr;
212         } else {
213             child = target->l_ptr;
214         }
215
216         if (fetch_bit(key, parent_q->bit_idx) == 0) {
217             parent_q->l_ptr = child;
218         } else {
219             parent_q->r_ptr = child;
220         }
221         delete target;
222         return true;
223     }
224
225     TrieNode<T>*t_dummy, *q_back, *q_back_p;
226     locate(q->k, t_dummy, q_back, q_back_p);
227
228     TrieNode<T>* q_child;
229     if (q->l_ptr == target) {
230         q_child = q->r_ptr;

```

```

231     } else {
232         q_child = q->l_ptr;
233     }
234
235     if (q_child == q) {
236         q_child = target;
237     }
238
239     if (parent_q) {
240         if (fetch_bit(key, parent_q->bit_idx) == 0) {
241             parent_q->l_ptr = q_child;
242         } else {
243             parent_q->r_ptr = q_child;
244         }
245     } else {
246         head->l_ptr = q_child;
247     }
248
249     if (q_back && q_back != q) {
250         if (fetch_bit(q->k, q_back->bit_idx) == 0) {
251             q_back->l_ptr = target;
252         } else {
253             q_back->r_ptr = target;
254         }
255     }
256
257     target->val = q->val;
258     target->k = q->k;
259     delete q;
260     return true;
261 }
262
263 int store(const std::string& path) {
264     std::ofstream os(path, std::ios::binary);
265     if (!os) {
266         std::cerr << "ERROR: " << std::strerror(errno) << "\n";
267         return -1;
268     }
269     if (head) {
270         write_recursive(os, head->l_ptr);
271     }
272     return 0;
273 }
274
275 int restore(const std::string& path) {
276     std::ifstream is(path, std::ios::binary);
277     if (!is) {
278         std::cerr << "ERROR: " << std::strerror(errno) << "\n";
279         return -1;

```

```

280     }
281     TrieMap<T> next;
282     size_t s_len;
283     while (is.read(reinterpret_cast<char*>(&s_len), sizeof(s_len))) {
284         std::string s_key(s_len, ' ');
285         T s_val;
286         if (!is.read(s_key.data(), s_len) || !is.read(reinterpret_cast<char*>(&
            s_val), sizeof(T))) {
287             return -1;
288         }
289         next.add(s_key, s_val);
290     }
291     std::swap(head, next.head);
292     return 0;
293 }
294 };
295
296 void normalize(std::string& s) {
297     size_t len = s.size();
298     for (size_t i = 0; i < len; ++i) {
299         if (s[i] >= 'A' && s[i] <= 'Z') {
300             s[i] |= 32;
301         }
302     }
303 }
304
305 int main() {
306     std::ios_base::sync_with_stdio(false);
307     std::cin.tie(NULL);
308
309     TrieMap<uint64_t> dictionary;
310     std::string cmd;
311
312     while (std::cin >> cmd) {
313         try {
314             if (cmd == "+") {
315                 std::string k;
316                 uint64_t v;
317                 std::cin >> k >> v;
318                 normalize(k);
319                 if (dictionary.add(k, v)) {
320                     std::cout << "OK\n";
321                 } else {
322                     std::cout << "Exist\n";
323                 }
324             } else if (cmd == "-") {
325                 std::string k;
326                 std::cin >> k;
327                 normalize(k);

```

```

328         if (dictionary.del_key(k)) {
329             std::cout << "OK\n";
330         } else {
331             std::cout << "NoSuchWord\n";
332         }
333     } else if (cmd == "!") {
334         std::string sub, path;
335         std::cin >> sub >> path;
336         if (sub == "Save") {
337             if (dictionary.store(path) == 0) {
338                 std::cout << "OK\n";
339             }
340         } else {
341             if (dictionary.restore(path) == 0) {
342                 std::cout << "OK\n";
343             }
344         }
345     } else {
346         normalize(cmd);
347         uint64_t* res = dictionary.find(cmd);
348         if (res) {
349             std::cout << "OK: " << *res << "\n";
350         } else {
351             std::cout << "NoSuchWord\n";
352         }
353     }
354 } catch (const std::bad_alloc&) {
355     std::cout << "ERROR: Memory failure\n";
356 } catch (const std::exception& e) {
357     std::cout << "ERROR: " << e.what() << "\n";
358 }
359 }
360 return 0;
361 }

```

3 Исследование качества программного обеспечения

1 Первичное профилирование и обнаружение узких мест

На начальном этапе разработки было проведено профилирование программы с помощью утилиты `gprof`. Результаты показали, что значительная часть времени (около 12% напрямую и до 45% косвенно через итераторы) тратилась на функцию `normalize`. Это было вызвано использованием стандартных итераторов строк и функции `std::tolower`, которая учитывает системные локали, что замедляет обработку каждого символа.

2 Оптимизация кода

На основе полученных данных были проведены следующие улучшения:

1. **Оптимизация `normalize`:** Итераторы были заменены на прямой доступ к элементам строки по индексу. Вместо медленной функции `tolower` был применен побитовый сдвиг (`s[i] |= 32`), что допустимо для символов латиницы и работает значительно быстрее.
2. **Использование `std::move`:** В основном цикле программы была добавлена семантика перемещения при передаче ключей в структуру данных, что исключило лишние аллокации памяти.
3. **Инлайнинг:** Критически важные функции (`fetch_bit`) были помечены как `inline`, что позволило компилятору встроить их код напрямую в места вызова при использовании флага `-O3`.

3 Повторное профилирование (`gprof`)

После внесения изменений было проведено повторное профилирование на тесте из 1,500,000 операций. Результаты Flat Profile:

% времени	self (сек)	вызовы	мс/вызов	имя функции
55.82	2.74	—	—	<code>main</code> (ввод и нормализация)
22.82	1.12	400,000	0.00	<code>TrieMap::add</code>
20.78	1.02	400,000	0.00	<code>TrieMap::del_key</code>
0.41	0.02	64	0.31	<code>frame_dummy</code>

Анализ результатов после оптимизации:

1. Функция `normalize` и вспомогательные методы работы с битами исчезли из топа профилировщика. Это означает, что они были успешно встроены (*inlined*) компилятором, либо их время выполнения стало пренебрежимо малым.

2. Основное время (55.8%) теперь сосредоточено в `main`, что включает в себя чтение данных из стандартного потока ввода и быструю нормализацию.
3. Операции добавления и удаления распределены практически поровну (22.8% и 20.8%), что подтверждает сбалансированность и предсказуемость дерева PATRICIA.

4 Анализ памяти (AddressSanitizer)

После оптимизации логики удаления (`del_key`) программа была повторно проверена инструментом `AddressSanitizer` для исключения ошибок сегментации и утечек памяти. Команда запуска:

```
g++ -std=c++20 -O3 -g -fsanitize=address main.cpp -o solution
./solution <test.txt
```

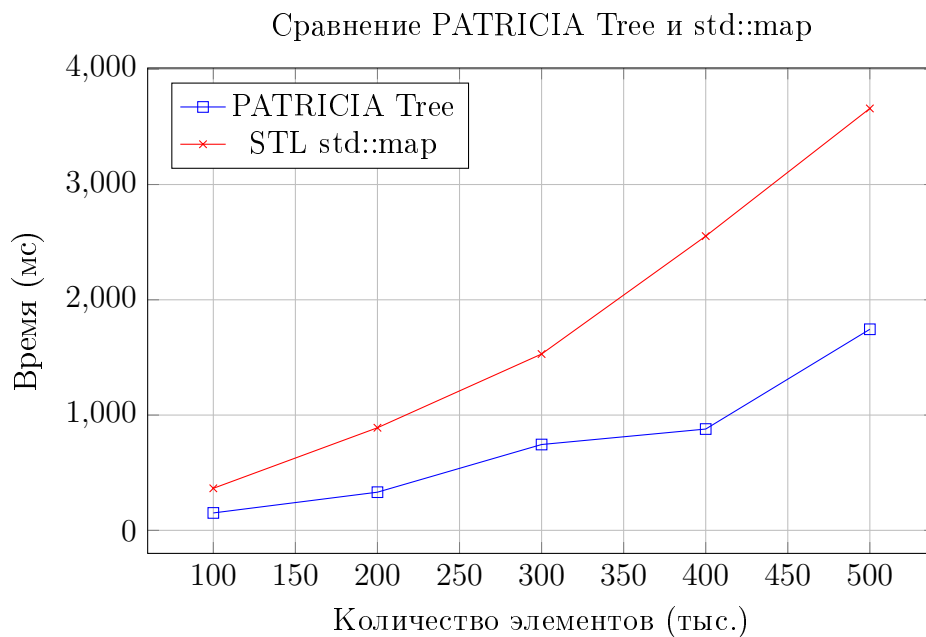
Результаты: Программа успешно обработала 1,500,000 команд без ошибок. Отчет `LeakSanitizer` показал: «0 bytes definitely lost». Все выделенные узлы дерева корректно очищаются в деструкторе через рекурсивный вызов `deep_clear`.

4 Тест производительности

Для оценки эффективности реализованной структуры данных было проведено сравнение с `std::map` из стандартной библиотеки. Тестирование проводилось на наборах данных объемом от 100 до 500 тысяч операций.

Кол-во элементов (тыс.)	PATRICIA Tree (мс)	STL <code>std::map</code> (мс)
100	150.976	364.560
200	331.039	889.933
300	744.557	1529.501
400	878.791	2552.388
500	1744.340	3659.964

Ниже представлен график зависимости времени выполнения от количества элементов:



5 Консоль

Ниже представлен протокол запуска тестов производительности:

```
root@5703d59f664e:/workspaces/c_labs/da_lab2# ./solution <test_100k.txt
Patricia Tree: 150.976 ms
```

STL std::map: 364.560 ms

root@5703d59f664e:/workspaces/c_labs/da_lab2# ./solution <test_200k.txt

Patricia Tree: 331.039 ms

STL std::map: 889.933 ms

root@5703d59f664e:/workspaces/c_labs/da_lab2# ./solution <test_300k.txt

Patricia Tree: 744.557 ms

STL std::map: 1529.501 ms

root@5703d59f664e:/workspaces/c_labs/da_lab2# ./solution <test_400k.txt

Patricia Tree: 878.791 ms

STL std::map: 2552.388 ms

root@5703d59f664e:/workspaces/c_labs/da_lab2# ./solution <test_500k.txt

Patricia Tree: 1744.340 ms

STL std::map: 3659.964 ms

6 Выводы

В ходе выполнения лабораторной работы была реализована структура данных PATRICIA Tree. Использование инструментов профилирования (**gprof**) позволило понять, что накладные расходы на работу со стандартными строками **std::string** сопоставимы с затратами на саму логику дерева. Анализ через **AddressSanitizer** подтвердил надежность управления памятью.

Список литературы

- [1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 2-е издание*. — Издательский дом «Вильямс», 2007. Перевод с английского: И. В. Красиков, Н. А. Орехова, В. Н. Романов. — 1296 с. (ISBN 5-8459-0857-4 (рус.))
- [2] *Сортировка подсчётом* — *Википедия*.
URL: http://ru.wikipedia.org/wiki/Сортировка_подсчётом (дата обращения: 16.12.2013).